

Fundamentos computacionais da tese

Redes neurais de pulso e lógica paraconsistente

André Furlan - ensismoebius@gmail.com

<https://github.com/ensismoebius>

2026

Fundamentos de redes neurais

Como uma rede neural é construída

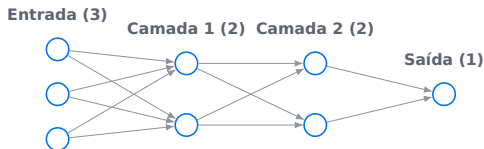
O problema: aprender a transformação em vez de programá-la à mão

Dado um sinal de entrada (um pixel, uma amostra de áudio, uma janela de EEG) e a resposta certa que se quer para ele, como chegar da entrada à resposta sem escrever a regra manualmente? A rede neural é essa transformação — com parâmetros que o próprio treino ajusta, em vez de uma fórmula fixada por quem programou.

Uma rede neural é organizada em **camadas**:
entrada → uma ou mais camadas ocultas → saída.

Cada neurônio recebe as saídas da camada anterior vezes seus **pesos** (parte linear), soma tudo, e aplica uma **ativação** (parte não linear).

Sem a não linearidade, empilhar camadas equivaleria a uma única camada linear — é a ativação que dá à rede o poder de aproximar funções complexas.



O que é um peso, e como ele é calculado

Um **peso** é um número real associado a uma conexão entre dois neurônios — é o parâmetro que a rede efetivamente **aprende**. Os dados de entrada não mudam; os pesos, sim.

Dentro de um neurônio, os pesos entram na **combinação linear**: cada entrada é multiplicada pelo seu peso, e os produtos são somados.

$$z = \sum_i w_i x_i \quad y = \sigma(z)$$

Exemplo numérico real com o primeiro neurônio da rede do slide anterior:

Entrada $x = [1,0, 0,5, -0,5]$

Pesos $w = [0,4, -0,3, 0,6]$.

$$z = 0,4 \cdot 1,0 + (-0,3) \cdot 0,5 + 0,6 \cdot (-0,5) = -0,05 \implies y = \sigma(-0,05) \approx 0,4875$$

Cada um dos outros neurônios repete exatamente esse cálculo, com sua própria linha de pesos. Antes do treino, os pesos partem de algum valor inicial (aqui, fixos); o que o treino faz é ajustá-los, um pouco a cada passo, para que a saída da rede se aproxime do que se quer.

Como uma rede neural é treinada

- 1. Avanço (Forward):** calcula a saída y a partir da entrada e dos pesos atuais.
- 2. Perda (loss):** compara y com o alvo, num único número que cresce com o erro.
- 3. Retropropagação (backward):** a regra da cadeia calcula $\partial L / \partial w$ — quanto a perda mudaria se cada peso mudasse um pouco.
- 4. Atualização:** cada peso anda um passo no sentido **contrário** ao gradiente.

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

Exemplo numérico real (neurônio com entrada: $x = 2,0$, alvo: $= 0,9$, $w_0 = -1,0$, $\eta = 3,0$):

$$z = w_0 x = -2,0 \Rightarrow$$

$$y = \sigma(z) \approx 0,1192 \Rightarrow$$

$$L = \frac{1}{2}(y - \text{alvo})^2 \approx 0,3048$$

$$\frac{\partial L}{\partial w} \approx -0,1640 \quad \Rightarrow \quad w_1 = -1,0 - 3,0 \cdot (-0,1640) \approx -0,508$$

Repetindo os quatro passos, y converge para o alvo. É exatamente esse ciclo que se questiona: E se os pesos só valerem $\{-1, 0, 1\}$, ou se a ativação virar um pulso discreto?

Demonstração ao vivo

Backpropagation, Rede 3-2-2-1

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.mlp
```

Demonstração ao vivo

Backprop → Forward e backward clássicos

► **Abrir no software**

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.classic
```

A camada inteira é uma multiplicação de matrizes

Os slides anteriores calcularam **um neurônio de cada vez**. Mas a camada toda é **uma única operação**: empilhe os pesos de cada neurônio como **linhas** de uma matriz W , e o produto escalar de cada neurônio passa a ser uma linha do produto Wx .

A regra do mapeamento é esta:

linha = neurônio de *destino*;

coluna = neurônio de entrada.

$$z = Wx \quad y = \sigma(z)$$

Exemplo numérico real, com a rede da demonstração ($2 \rightarrow 2 \rightarrow 1$):

$$\underbrace{\begin{bmatrix} 0,8 & -0,4 \\ -0,5 & 0,9 \end{bmatrix}}_{W_1} \underbrace{\begin{bmatrix} 0,9 \\ 0,4 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 0,56 \\ -0,09 \end{bmatrix}}_z \xrightarrow{\sigma} \underbrace{\begin{bmatrix} 0,6365 \\ 0,4775 \end{bmatrix}}_y$$

A primeira linha do resultado é $0,8 \cdot 0,9 + (-0,4) \cdot 0,4 = 0,56$: o mesmo $\sum_i w_i x_i$ do slide anterior, agora lido como “linha vezes vetor” (Conhecido como valor **z**). Já a ativação é um passo **separado**: σ age em cada célula isoladamente, sem misturar linhas — ao contrário da multiplicação, que existe justamente para combiná-las.

O backward é a mesma matriz, transposta

No forward, W_2 levou 2 valores (os ocultos) para 1 (a saída). O gradiente precisa fazer o caminho **inverso**: 1 volta para 2. A matriz que faz isso é a **mesma**, só que transposta: o que era linha vira coluna.

O erro volta pelos próprios pesos do forward, não por pesos novos.

$$W \leftarrow W - \eta \partial L / \partial W$$

$$\frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial z} \qquad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \otimes \text{entrada}$$

A regra da cadeia diluída nessas matrizes é o produto das derivadas locais de cada elo do caminho $x_1 \rightarrow H_1 \rightarrow O \rightarrow L$:

$$\frac{\partial L}{\partial w_{11}} = \underbrace{(-0,3173)}_{\partial L / \partial y_O} \cdot \underbrace{0,2432}_{\sigma'(z_O)} \cdot \underbrace{1,2}_{w_{H_1 O}} \cdot \underbrace{0,2314}_{\sigma'(z_{H_1})} \cdot \underbrace{0,9}_{x_1} = -0,01928$$

Demonstração ao vivo

Backprop → A rede como matrizes

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.matrix
```

Demonstração ao vivo

Backprop → Camadas e a regra da cadeia

► **Abrir no software**

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.chain
```

Percorra os passos de $\partial L / \partial a_2$ até $\partial L / \partial b_1$: cada carta é a derivada local de um bloco, na coluna do bloco. Feche com os cinco fatores em fila.

Redes de pulso

Redes neurais de pulso (RNP)

Por que pulsos?

Uma RNA convencional ativa **todos** os neurônios em **toda** passada — o custo energético não depende do conteúdo do dado, só do tamanho da rede.

O neurônio biológico só dispara quando o potencial acumulado ultrapassa um limiar; fica quieto o resto do tempo.

Maass (1997) chamou as RNPs de "*modelos de redes neurais **terceira geração***" (MAASS, 1997) — depois do perceptron (1ª geração) e das redes com ativação contínua/sigmoide (2ª geração).

Redes neurais de pulso (RNP)

Uma ressalva importante

Uma RNP **não** é uma simulação um-para-um de neurônios biológicos. Ela aproxima certas capacidades computacionais de propriedades biológicas específicas — não o neurônio inteiro.

Modelos mais fiéis à biologia existem (Hodgkin-Huxley (GERSTNER et al., 2014), dendritos não-lineares), mas são caros demais para treinar em escala. As RNPs usadas em aprendizado de máquina adotam modelos simplificados, como o **LIF** (próximo slide).

RNA vs. RNP: o par que se confunde

Rede Neural Artificial (RNA)

Ativação: valor **contínuo** por neurônio, por camada.
Processamento **síncrono**: uma passada completa por amostra.
Sem dimensão temporal intrínseca (salvo recorrente).
Custo computacional fixo, independente da atividade.

Rede Neural de Pulso (RNP)

Ativação: **pulso** (0 ou 1), esparsos no tempo.
Processamento **orientado a evento**: só computa quando há pulso.
Dinâmica temporal **intrínseca**: potencial de membrana acumula.
Informação em **taxa** e/ou **tempo** de disparo (KASABOV, 2019).

Consequência prática

Uma RNP processa naturalmente **fluxos** (áudio, EEG, câmera de eventos) — um dado estático exige “esticá-lo” por vários passos de tempo, custo que a RNA não tem.

Demonstração ao vivo

SNN → Normalização: por característica x por janela

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.normalization
```

Áudio padroniza cada característica com média/desvio ajustados uma vez no treino (CMVN); EEG recalcula a cada janela, e uma deriva de $+50 \mu\text{V}$ some. Só o caminho ajustado pode vazar dados de teste – em silêncio.

Demonstração ao vivo

SNN → Sinal e spikes

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.spikes
```

O neurônio *Leaky Integrate-and-Fire* (LIF)

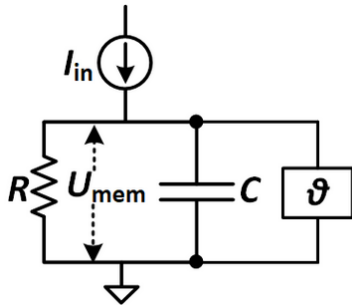
O modelo RC

O LIF se comporta como um circuito Resistor-Capacitor (RC).

R : resistência de vazamento; C : capacitância; V_{mem} : potencial acumulado; ϑ : interruptor que descarrega o capacitor (dispara um pulso) ao atingir o limiar.

Governado por (ESHKAGHIAN et al., 2023):

$$\tau \frac{dV_{mem}}{dt} = R I_{in}(t) - V_{mem}(t), \text{ com } \tau = R \cdot C .$$



Fonte: (ESHKAGHIAN et al., 2023)

O neurônio *Leaky Integrate-and-Fire* (LIF)

Exemplo numérico: decaimento sem entrada

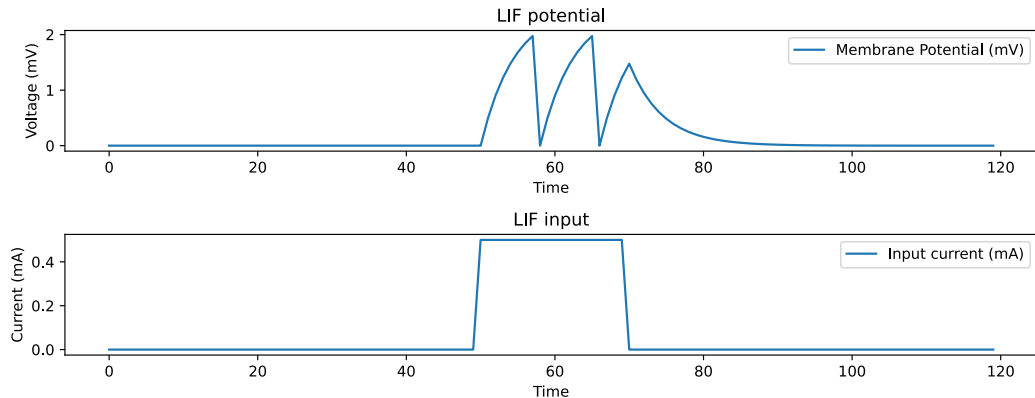
Com $R = 5 \Omega$, $C = 1 F$ (logo $\tau = R \cdot C = 5$) e potencial inicial $V_{mem}(0) = 1$, sem corrente de entrada:

$$V_{mem}(t) = V_{mem}(0) \cdot e^{-t/\tau} \implies V_{mem}(1) = e^{-1/5} \approx 0,8187$$

O potencial **vaza** exponencialmente na ausência de estímulo — daí o nome *leaky*. Quando um estímulo chega, o potencial sobe (mesma matemática, sinal oposto) até cruzar um limiar V_{th} , disparando um pulso e reiniciando.

O neurônio *Leaky Integrate-and-Fire* (LIF)

Comportamento completo simulado



Corrente de 0,5 mA injetada entre os passos 51 e 70. Fonte: elaborado pelo autor.

Demonstração ao vivo

SNN $\rightarrow$ LIF

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.lif
```

Os parâmetros padrão ($\tau=5$, $R=5$, $V_{th}=1$) já reproduzem o exemplo numérico anterior; aperte Play para o comportamento completo.

time_steps e Δt : quantos passos e quanto dura cada passo

Um neurônio de pulso guarda estado entre atualizações. Para simular essa memória, é preciso saber tanto **quantas atualizações** fazer quanto **quanto tempo simulado** decorre entre elas.

time_steps (T)		Δt
Responde a	Quantos quadros?	Quanto dura cada quadro?
Exemplo	$T = 16$: 16 atualizações	$\Delta t = 1$: 1 unidade por quadro

Duração simulada: $T\Delta t$. Com $T = 16$, mudar Δt de 1 para 16 muda a duração de 16 para 256 unidades, sem mudar o número de quadros.

No LIF, a fração de potencial que sobrevive a cada passo é $\beta = e^{-\Delta t/(RC)}$. Com $RC = 5$, $\beta = 0,82$ para $\Delta t = 1$, mas cai para 0,04 se $\Delta t = 16$: a unidade quase esquece o estado anterior.

Por que importa? Confundir as duas grandezas pode alterar silenciosamente a memória temporal. E, ao empilhar sequências no lote, a ordem dos quadros também precisa continuar correta (GERSTNER et al., 2014; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN $\rightarrow$ time_steps x delta_t

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.timesteps
```

Quantos quadros x quanto dura um quadro – e por que a ordem das linhas no tensor (T*B,F) importa.

Codificação Poisson: intensidade vira chance

Uma rede precisa receber números comuns, mas seus neurônios trocam pulsos. Na codificação Poisson, o valor de entrada não determina um pulso certo: determina a **probabilidade** de disparar em cada passo.

Entrada	Probabilidade por passo	Leitura após 10 passos
$x = 0,6, r_{\max} = 0,9$	$p = x r_{\max} = 0,54$	$E[n] = 10p = 5,4$ pulsos

Duas repetições com a mesma entrada podem gerar contagens diferentes; ambas tendem a representar a mesma intensidade.

Por que importa? A contagem aproxima o valor, mas traz variação aleatória. Mais passos tornam a estimativa de taxa mais estável, ao custo de mais tempo de simulação.

É uma escolha de representação, não ruído adicionado depois do treino. A codificação por taxa é comum, mas não é a única forma de representar informação em pulsos (DIEHL et al., 2015; ESHRAGHIAN et al., 2023).

O que observar na demonstração

O mesmo sinal alimenta cada passo. A intensidade define a chance de disparo; não garante que um pulso apareça em um instante específico.

Demonstração ao vivo

SNN → Codificação Poisson

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.poisson
```

Demonstração ao vivo

SNN → Codificação Poisson (imagem)

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.poisson_image
```

Cada codificação tem seu próprio piso de erro

Mesmo antes do treino, converter um valor contínuo em pulsos pode perder precisão. Esse **piso de codificação** não é ruído do sensor nem erro do classificador: é o erro que a representação e sua leitura já permitem.

Exemplo idealizado: x uniforme em $[0, 1]$, $T = 16$ passos e a mesma métrica (erro RMS de reconstrução):

Codificação	De onde vem o erro	Piso RMS
Direta	x entra sem quantização em pulsos	0
Latência	arredondamento para um dos 16 instantes	$\frac{1}{15\sqrt{12}} \approx 0,019$
Poisson	contagem aleatória de pulsos	$\sqrt{\frac{1}{6 \cdot 16}} \approx 0,102$

O zero da codificação direta vale só para esta etapa ideal de entrada e reconstrução; não significa que a rede inteira terá erro zero.

Por que importa? Se um modelo começa com um piso maior, comparar apenas seu erro final pode atribuir à arquitetura uma diferença que veio da codificação.

Compare as redes com o mesmo T , a mesma métrica e em relação ao piso de cada codificação. A escolha entre códigos também envolve latência, energia e robustez, não só erro (ESHRAGHIAN et al., 2023; LIU; LEUNG; DRAGOTTI, 2023).

Demonstração ao vivo

SNN → Ruído estrutural da codificação

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.encoding_noise
```

“Direta é latência é Poisson” pode ser só o piso de ruído estrutural de cada codificação, não uma diferença real de informação.

A perda precisa medir onde o código guarda a informação

Uma função de perda só consegue corrigir aquilo que mede. Se o código guarda o valor no **instante** do pulso, contar pulsos pode declarar sucesso mesmo quando o pulso chega na hora errada.

	Alvo	Previsão
Instante do pulso, $T = 16$	quadro 4	quadro 12
Contagem de pulsos	1	1

Perda de contagem: $(1 - 1)^2 = 0$; o treino diz “perfeito”, embora o pulso esteja 8 quadros atrasado.

Perda de tempo: $(12 - 4)^2 = 64$; ela enxerga o erro que a codificação por latência realmente representa.

Há outra armadilha: se a unidade nunca dispara, esta implementação atribui $t = T = 16$. A perda de tempo vale $(16 - 4)^2 = 144$, mas seu *backward* não escreve gradiente para $t = T$; a unidade permanece silenciosa.

Importância prática

Confirme que codificação, alvo e perda concordam; monitore se há gradientes vivos. Uma perda baixa, sozinha, não prova que a rede aprendeu a resposta temporal (ESHRAGHIAN et al., 2023; LIU; LEUNG; DRAGOTTI, 2023).

Demonstração ao vivo

SNN → Perda incompatível com a codificação

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.encoding_loss_mismatch
```

Perda de contagem sobre latência: o treino reporta perda zero sem corrigir nada.

Perda certa (SpikeTimeLoss): uma unidade que nunca dispara fica sem gradiente. Duas falhas silenciosas – e a validação que as torna barulhentas.

Como treinar uma função que não tem gradiente?

Quatro famílias de resposta, historicamente

A função de disparo do LIF se aproxima de uma **função em degrau** (Heaviside): dispara ou não dispara, sem meio-termo. Sua derivada é zero em quase todo ponto — a descida de gradiente padrão não tem o que propagar (KASABOV, 2019).

Quatro famílias de resposta, historicamente:

STDP: regra local — se o neurônio pré-sináptico dispara **antes** do pós-sináptico, a conexão se fortalece; se depois, se enfraquece. Biologicamente plausível, não supervisionada.

Gradiente substituto + BPTT: usa a função em degrau real no *forward*, e uma aproximação suave e diferenciável só no *backward*.

Algoritmos evolutivos: seleção dos hiperparâmetros mais aptos ao longo de gerações.

Reservatório / computação dinâmica: treina só a camada de saída, mantendo pesos recorrentes fixos (*echo state networks*, *liquid state machines*).

Como treinar uma função que não tem gradiente?

Alternativa: converter em vez de treinar

Treina uma **RNA convencional**, com técnicas padrão de otimização. **Converte** os pesos treinados para uma RNP, balanceando pesos e limiares de disparo (DIEHL et al., 2015; RUECKAUER et al., 2017). Evita o problema do gradiente por completo — mas herda limitações: geralmente precisa de **mais passos de tempo** para aproximar a acurácia da RNA original, e nem toda arquitetura converte bem.

Demonstração ao vivo

SNN → Surrogate gradient

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.surrogate
```

Percorra os checkpoints "A derivada real" até "O gradiente substituto".

BPTT (1/4): um neurônio vê um filme, não uma foto

Uma rede comum calcula a saída de uma vez. Um neurônio LIF mantém parte do potencial anterior; por isso, a resposta de agora depende também do que aconteceu antes. Para treinar, a sequência é aberta em passos ligados pela memória.

Exemplo didático: $u_t = 0,5u_{t-1} + x_t$, limiar $V_{th} = 1$, potencial inicial $u_0 = 0$; ao disparar, o neurônio reinicia.

Passo	x_t	Cálculo de u_t	u_t antes do reset	Pulso?
1	0,8	$0,5(0) + 0,8$	0,8	Não
2	0	$0,5(0,8) + 0$	0,4	Não
3	0,8	$0,5(0,4) + 0,8$	1,0	Sim; reinicia

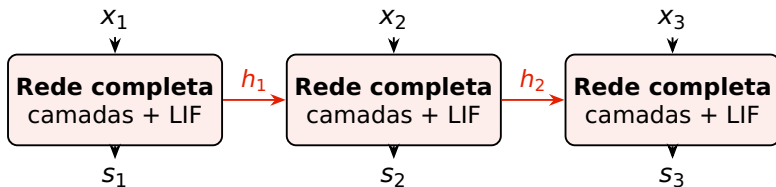
A intuição

O primeiro quadro ainda contribui no terceiro: sem ele, u_3 seria só 0,8 e não haveria pulso. Treinar apenas cada quadro isoladamente apagaria essa dependência temporal.

BPTT (2/4): abrir a rede ao longo do tempo

O problema: a saída de uma SNN depende do estado que veio antes. Para enxergar essa dependência, desenhamos a mesma rede uma vez por instante da sequência:

Desenho expandido: mesmas três cópias temporais, pesos compartilhados W



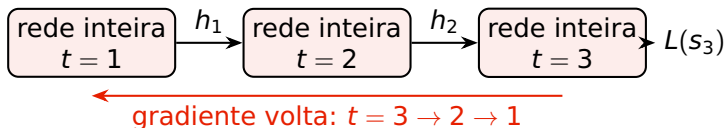
Cada caixa faz um passo da **rede inteira**; h_t é o estado interno que ela entrega ao passo seguinte. s_t é a saída naquele instante.

São três quadros do mesmo cálculo, não três redes independentes: todas atualizam os mesmos pesos W .

No exemplo anterior, $h_1 = 0,8$ chega ao segundo passo; após a entrada $x_3 = 0,8$, a memória faz o potencial chegar a $u_3 = 1,0$ e disparar.

BPTT (3/4): o erro volta pela rede aberta

No treino, primeiro calculamos a sequência para a frente. Depois, a regra da cadeia percorre o mesmo desenho para trás: o erro da saída em $t = 3$ pode atravessar o estado h_2 e chegar ao cálculo feito em $t = 1$.



Cada instante calcula uma contribuição para ajustar o mesmo peso; elas se somam:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^3 \frac{\partial L}{\partial W^{(t)}} = 0,10 - 0,05 + 0,20 = 0,25.$$

O gradiente que chega a um passo inclui tanto o erro local quanto o que retorna dos passos seguintes: é assim que o primeiro instante pode ser corrigido por ter influenciado o terceiro (WU et al., 2018; ESHRAGHIAN et al., 2023).

BPTT (4/4): o que o treino precisa guardar

	Rede feedforward	SNN + BPTT
Erro volta por	camadas	camadas e tempo
Estados guardados	uma passada	ao longo de T
Memória cresce com	profundidade	profundidade e T

Mais passos revelam dependências mais longas, mas aumentam computação e memória de treino.

O pulso real é um degrau, cuja derivada é zero quase em toda parte. O gradiente substituto aproxima essa derivada **só na volta**; na ida, o disparo continua discreto.

BPTT percorre o tempo; o gradiente substituto permite atravessar o degrau durante o treino.

Referência: (ESHRAGHIAN et al., 2023).

tdBN: colocar a corrente na escala do limiar

BatchNorm comum padroniza valores para média 0 e variância 1. Isso funciona em redes contínuas, mas um neurônio de pulso só dispara quando seu potencial alcança V_{th} ; uma escala fixa pode deixar toda a camada abaixo desse limiar.

	BatchNorm comum	tdBN, $V_{th} = 2, \alpha = 1$
Entrada $X = [0, 2, 4, 6]$	$[-1,34, -0,45, 0,45, 1,34]$	$[-2,68, -0,89, 0,89, 2,68]$
Escala do desvio	1	$\alpha V_{th} = 2$
Valores acima do limiar	0 de 4	1 de 4

A tdBN usa $Y = \alpha V_{th}(X - \mu)/\sigma$: a escala acompanha o limiar, em vez de ignorá-lo.

Por que importa? Mais valores podem alcançar o limiar e gerar gradientes úteis; isso ajuda a treinar redes de pulso profundas e reduz o problema de camadas sem disparos.

Não é garantia de que cada neurônio disparará nem substitui uma boa escolha de dinâmica e perda. É uma normalização proposta para estabilizar o treinamento direto de SNNs profundas (ZHENG et al., 2021; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → Normalização dependente de limiar (tdBN)

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.tdbn
```

BatchNorm comum fixa variância 1 e, com $V_{th} = 2$, cala a camada; tdBN espalha a corrente em $\alpha \cdot V_{th}$, e a fração acima do limiar deixa de depender de V_{th} .

Regularizar a taxa: evitar silêncio e rajadas

Uma rede pode acertar a saída disparando quase nunca ou disparando em quase todo quadro. Uma penalidade de taxa acrescenta ao objetivo de treino um custo quando a atividade média da camada sai de uma faixa escolhida:

$$L_{\text{reg}} = \lambda \left[\max(0, r_{\min} - \rho)^2 + \max(0, \rho - r_{\max})^2 \right], \quad \rho = \text{taxa média da camada.}$$

Exemplo: faixa $[0,10, 0,80]$. Uma camada com média 0,02 recebe pressão para disparar mais; uma com média 0,95 recebe pressão para disparar menos.

Por que importa? Controlar a atividade pode favorecer pulsos esparsos e equilibrar a taxa com a tarefa. Isso pode reduzir operações em hardware orientado a eventos, mas a penalidade não garante economia em qualquer processador.

Limite importante: se duas unidades têm taxas 0,02 e 0,95, a média é 0,485; a penalidade sobre a média é zero. Esta formulação vê a camada, não cada unidade.

A forma da penalidade e a faixa desejada são escolhas de projeto, não uma regra universal para toda SNN (YAN et al., 2022; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → Regularização de taxa de disparo

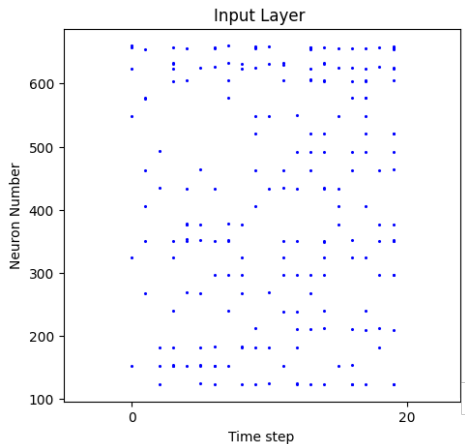
► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.firing_rate_reg
```

Um neurônio quase morto e um em rajada são empurrados de volta à faixa alvo de taxa de disparo; dentro da faixa, a penalidade não faz nada.

Esparsidade: onde mora a eficiência



Eixo horizontal: tempo. Eixo vertical: índice do neurônio. Fonte:
elaborado pelo autor.

Na maior parte do tempo, a **grande maioria** dos neurônios está silenciosa. Pulsos são representados como **uns** dispersamente distribuídos em uma sequência de **zeros** (ESHRAGHIAN et al., 2023). Um neurônio LIF bem parametrizado tolera ruído: age como um **filtro passa-baixa**, só disparando quando o sinal se acumula de forma sustentada.

Esparsidade: onde mora a eficiência

A promessa, com precisão

Esparsidade só vira economia de energia real em hardware que sabe **pular** o zero. Em GPU/TPU convencional, a esparsidade é invisível ao hardware — ele computa a multiplicação por zero do mesmo jeito.

Volta-se a este ponto na próxima seção, sobre hardware neuromórfico.

TrueNorth (IBM, 2014): 1 milhão de neurônios, chip totalmente assíncrono e orientado a evento (MEROLLA et al., 2014).

Loihi (Intel, 2018): aprendizado *on-chip*, arquitetura *manycore* orientada a evento (DAVIES et al., 2018).

SpiNNaker (Manchester, 2014): milhões de núcleos ARM emulando pulsos em tempo real biológico (FURBER et al., 2014).

O modo de falha silencioso

O ganho energético de uma RNP só se realiza **nesse hardware especializado**. Rodada em GPU/TPU convencional — otimizada para multiplicação densa, não para pular zeros — simular T passos de tempo por amostra pode deixar a RNP **mais lenta** que a RNA equivalente.

Câmeras de eventos (DVS): sensores que já emitem pulsos nativamente — casamento direto com RNPs.

EEG/BCI: sinais biológicos já são fluxos temporais esparsos por natureza — domínio onde a codificação temporal da RNP é vantagem, não custo extra. Para o formalismo de modelos de pulso em neurociência, ver (GOODMAN et al., 2022).

Computação de borda: dispositivos vestíveis e embarcados, onde bateria é o recurso mais escasso.

Spikformer (ZHOU et al., 2023): reformula o mecanismo de atenção do *transformer* em operações compatíveis com pulsos, aproximando as duas linhagens de arquitetura que dominam *deep learning* hoje.

Panorama mais amplo do campo em (ROY; JAISWAL; PANDA, 2019).

Demonstração ao vivo

Comparação → Autoencoders (SNN x LSTM x GRU x Transformer)

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh comparison.autoencoders
```

Como o Meeting01 compara quatro famílias sem trapacear: mesmo gargalo, mesmo alvo, referências média e PCA. A PCA é real (janelas sintéticas); as famílias ficam sem números inventados – só o experimento LOSO os preenche.

Lógica paraconsistente

Por que não só treinar e comparar?

O problema: escolher entre muitos extratores de características

A tese compara extração manual (*wavelet*) com extração automatizada (*autoencoder*) — 24 variantes de *autoencoder*, mais as manuais, para dois sinais. Treinar um classificador para **cada** combinação só para descobrir qual representação é melhor é caro, e pior: o resultado fica amarrado a **qual** classificador se escolheu treinar.

A alternativa: medir a qualidade do vetor de características **antes** de treinar qualquer coisa, olhando só para sua geometria — o quanto cada classe se agrupa (compactidade intraclasse, α) e o quanto classes diferentes se misturam (sobreposição interclasse, β).

O plano paraconsistente

α e β definem um ponto num plano com quatro cantos: **Verdade** (classes compactas e separadas — o alvo), **Falsidade**, **Ambiguidade** e **Indefinição**. Quanto mais perto de Verdade, melhor a representação separa as classes **sem treinar classificador nenhum**.

As próximas demos mostram esse plano, uma falha dele que passou despercebida até ser medida, e a correção — a contribuição central desta tese.

Demonstração ao vivo

Paraconsistente → Plano paraconsistente

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.plane
```

Alpha (compacidade intraclasses) e beta (sobreposição interclasses) definem um ponto no plano, com quatro cantos: Verdade, Falsidade, Ambiguidade, Indefinição.

Demonstração ao vivo

Paraconsistente → D_truth x D_penalized

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.dpenalized
```

Por D_truth, um extrator "morto" ($\sqrt{2} \approx 1,41$) passa à frente de um fraco porém real (1,50); D_penalized leva os três cantos ruins a exatamente 2 e fecha essa brecha – contribuição central da tese do autor.

Demonstração ao vivo

Paraconsistente → Busca genética de arquiteturas
(extensão)

► Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.ga_pareto
```

Extensão fora da monografia: o software/nn usa D_penalized numa busca NSGA-II. A fronteira de Pareto reúne arquiteturas em que nenhuma é melhor em tudo; a regra de Deb descarta as que excedem o teto de latência.

Desafios e síntese

Limitações do experimento

A limitação dominante: tamanho da base

15 locutores, 1974 amostras. Sob protocolo de verificação — partições disjuntas por locutor — cada dobra treina com poucas identidades: risco alto de o classificador não ter sinal suficiente para generalizar.

As três categorias de representação (manual, *autoencoder*, fusão) **não** foram comparadas por um classificador comum — só pelo critério paraconsistente.

Os *autoencoders* tiveram $\bar{\beta} > 0,97$ em toda configuração: latentes perto do colapso. Não dá para distinguir, com os dados disponíveis, “extrator pior” de “treino insuficiente”.

Consequência

A seleção paraconsistente ranqueia qualidade **antes** de treinar um classificador — mas ranquear bem um extrator degenerado via treino insuficiente ainda é um risco que $D_{\text{penalizado}}$ por si só não resolve.

Extração manual venceu a seleção paraconsistente para voz e EEG, com a mesma configuração (*wavelet* Haar, LFCC) — nenhuma das 24 variantes de *autoencoder* se aproximou.

Autenticação ficou perto do acaso: melhor EER = 0,4534, melhor AUC = 0,5452, contra 0,5/0,5 de um classificador aleatório — um resultado negativo, não um ajuste fino pendente.

EEG e fusões superaram voz isolada entre as configurações mais bem colocadas — consistente com a motivação de complementar fala degradada com sinal cerebral.

Três contribuições metodológicas

$D_{\text{penalizado}}$ — fecha a brecha dos três cantos degenerados do plano paraconsistente.

Diagnóstico do piso de ruído estrutural de codificações temporais (*direct* $\succ$ *latency* $\succ$ *poisson*).

Protocolo em duas fases: seleção do extrator separada da medição de desempenho.

Obrigado.

André Furlan — ensismoebius@gmail.com

 DAVIES, M. et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, v. 38, n. 1, p. 82–99, 2018.

 DIEHL, P. U. et al. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In: *2015 International Joint Conference on Neural Networks (IJCNN)*. [S.l.]: IEEE, 2015. p. 1–8.

 ESHRAGHIAN, J. K. et al. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, v. 111, n. 9, p. 1016–1054, 2023.

 FURBER, S. B. et al. The spinnaker project. *Proceedings of the IEEE*, v. 102, n. 5, p. 652–665, 2014.

Referências II

-  GERSTNER, W. et al. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014. ISBN 9781107060838. Disponível em: <https://neurondynamics.epfl.ch/online/Ch2.S2.html>.
-  GOODMAN, D. et al. *Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022*. Zenodo, 2022. Disponível em: <https://doi.org/10.5281/zenodo.7044500>.
-  KASABOV, N. K. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. [S.l.]: Springer, 2019.
-  LIU, S.; LEUNG, V. C. H.; DRAGOTTI, P. L. First-spike coding promotes accurate and efficient spiking neural networks for discrete events with rich temporal structures. *Frontiers in Neuroscience*, v. 17, p. 1266003, 2023.

-  MAASS, W. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, v. 10, n. 9, p. 1659–1671, 1997.
-  MEROLLA, P. A. et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, v. 345, n. 6197, p. 668–673, 2014.
-  ROY, K.; JAISWAL, A.; PANDA, P. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, v. 575, n. 7784, p. 607–617, 2019.
-  RUECKAUER, B. et al. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, v. 11, p. 682, 2017.

Referências IV

 WU, Y. et al. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, v. 12, p. 331, 2018.

 YAN, Y. et al. Backpropagation with sparsity regularization for spiking neural network learning. *Frontiers in Neuroscience*, v. 16, p. 760298, 2022.

 ZHENG, H. et al. Going deeper with directly-trained larger spiking neural networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2021. v. 35, n. 12, p. 11062–11070.

 ZHOU, Z. et al. Spikformer: When spiking neural network meets transformer. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2023.